

Technical Report — Fact Attack: Multilingual NLI over Swiss Voting Booklets

- **Track:** Track 2A — OST: Multilingual Natural Language Inference over Swiss Official Voting Booklets
- **Event:** Hack Apertus Online (October 1–16, 2026)
- **Team:** Team Fact Attack 2026 — Josip Jukic, Felipe Wüthrich
- **Repository:** [GitHub](#) · **Experiment log:** [docs/experiments.md](#)
- **Demo:** Streamlit app (make web: claim check with highlighted evidence pages, benchmark dashboard) and the entailment CLI (`python -m src run -i cases.jsonl -o predictions.jsonl`, also inside the Docker image, see 2.5)

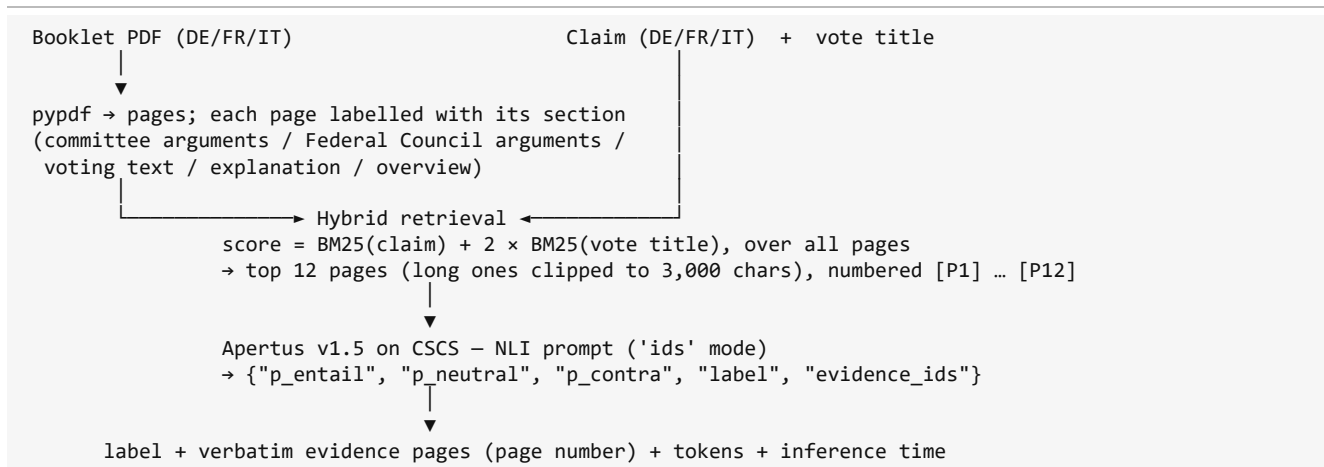
1. Summary

Swiss voters receive an official booklet for every federal vote, and campaigns make claims about it that are hard to check. We built a multilingual claim-verification system on **Apertus v1.5**: given a booklet (German, French or Italian) and a claim (any of the three languages), it classifies the claim as **Entailment (0)**, **Neutral (1)** or **Contradiction (2)** and returns the booklet pages that justify the decision, together with token usage and inference time.

On a test split of **5 voting dates never used during development** (402 pairs from the official dataset), the system reaches **macro-F1 0.940** on the advanced task (booklet PDF) and **0.975** on the beginner task (reference text), at ~5,600 / ~2,500 input tokens, ~57 output tokens and a p95 inference time of 2.9 s / 1.7 s. Over all 1,495 pairs of the dataset the advanced task reaches 0.946.

The central finding for the challenge's research question: **sending a dozen well-chosen pages beats sending the whole booklet** — 0.946 vs. 0.730 macro-F1 on the same 150 test pairs, with ~10× fewer input tokens.

2. Architecture



2.1 Document processing

`pdf_parser.py` extracts the text of every page with `pypdf` and detects section headings in all three languages ("Argumente Initiativkomitee", "Arguments du Conseil fédéral et du Parlement", "Il testo in votazione", ...). A heading applies to its page and the following pages until the next heading, and table-of-contents entries (heading followed by a page number) are ignored. Section labels matter because booklets deliberately contain opposing viewpoints: the model has to know whether a page states the committee's or the Federal Council's position.

2.2 Context selection: hybrid retrieval

`retriever.py` scores every page of the booklet with normalised BM25 against the claim plus twice the normalised BM25 against the vote title, and keeps the top 12 pages. Pages longer than 3,000 characters are clipped to the window that best matches claim and vote title (plus the page heading).

- The **vote title is written in the booklet's language**, so it anchors the right proposal even when the claim is in another language; lexical matching of the claim alone fails for cross-lingual pairs.
- Our first version filtered hard to one detected proposal. That dropped the per-proposal overview pages at the front of the booklet, where the gold passage sits for ~11 % of pairs, and chose the wrong proposal for

another ~10 %. Booklet-wide scoring raised gold-section recall on the dev split from 0.74 to 0.93.

- **Speaker boost:** if the claim names a side ("the initiative committee argues ...", "selon le Conseil fédéral ..."), the two best argument pages of that side are always among the 12 pages, replacing the lowest-ranked others. Without it, the committee's own page was missing in 15 of 36 committee-claim errors.

2.3 Apertus as the NLI model

The selected pages are numbered [P1]...[P12]. The system prompt defines the three labels, tells the model to judge only against the passages (not world knowledge), to answer Neutral for topics or details the passages do not address, to check claims attributed to an actor only against that actor's text, and to read "recommends rejecting X" as supported when the Federal Council and Parliament recommend "No". Apertus answers with a small JSON object: three confidences, the label, and the ids of the pages that justify it.

- **Evidence by id** guarantees verbatim booklet text with the correct page number and avoids paraphrased quotes.
- **Short answers:** ~55 output tokens instead of ~150 for a JSON answer with reasoning and quotes, which more than halves inference time (p95 5.5 s → 1.8 s on the dev split) at equal or better F1.
- For **Neutral**, the evidence list is empty, as required by the output format. If the model predicts 0 or 2 without citing a page, the top-ranked page is returned as evidence.
- **Decision rules:** in ~2 % of answers the label and the stated confidences disagree. The label is right 3× as often, so it is kept (Rule 0) — except a Neutral label next to a confident other relation, which was right in only 1 of 18 cases and follows the confidences (Rule 0b). Otherwise label and confidences agree and are used as given.

2.4 Beginner task

The reference string (up to 25k characters) is split into passages of ≤ 700 characters at sentence boundaries, numbered and sent with the same prompt, so evidence is a specific passage rather than the whole reference.

2.5 CLI and output format

`python -m src run -i cases.jsonl -o predictions.jsonl` reads the official case format (JSON or JSONL; a case has `booklet.path + vote` for the advanced task or `reference.text` for the beginner task) as well as rows of the OST dataset on Hugging Face (`claim, reference_string, booklet_url, vote`; also Parquet or CSV), and writes one record per case:

```
{"id": "case-0042", "label": 2, "label_name": "contradiction",  
  "evidence": [{"page": 7, "text": "<verbatim page text>"}],  
  "metrics": {"input_tokens": 5512, "output_tokens": 58, "inference_time_ms": 1934}}
```

Evidence is empty for Neutral. The claim language is detected automatically, and a failing case is reported on stderr and returned as a valid Neutral record instead of aborting the batch. `python -m src predict` checks a single claim.

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-70B-thinking` (the model our CSCS key is authorised for; the thinking phase is not triggered by our prompt, answers start directly with the JSON object).
- **Where it runs:** CSCS inference endpoint (`https://api.inference.cscs.ch/v1`), OpenAI-compatible chat completions, temperature 0, retries with exponential backoff.
- **Role:** Apertus is the only model in the pipeline. It performs the NLI classification and selects the evidence pages; retrieval is lexical (BM25) and runs locally. Apertus 8B v1.5 was used only for a comparison run (5.3); no other model was used for development or evaluation.

4. Data and evaluation protocol

- **Dataset:** `OSTswiss/MNLIOverSwissVotingBooklets` — 1,495 human-annotated claim/reference pairs from 60 booklets (2020–2026), balanced labels, ~66 % cross-lingual pairs.
- **Booklets:** downloaded from `bk.admin.ch` via the `booklet_url` field (`python -m src.hf_dataset`).
- **Split by voting date**, so all three language versions of a booklet stay on the same side:
- **dev** (1,093 pairs) for all design decisions;

- **test** (402 pairs, 5 voting dates: 2021-06-13, 2021-09-26, 2022-09-25, 2024-06-09, 2024-09-22), used only for before/after measurements. The two booklets our first version was tuned on are pinned to dev.
- **Metrics** (mirroring the judging criteria): macro-F1 over the three labels (primary), macro-F1 per claim→booklet language pair, share of cited evidence inside the gold reference section, input/output tokens, mean and p95 inference time. Every run is saved with its configuration and git commit in `results/`.

5. Results

5.1 Final configuration — full test split (402 pairs)

Task	Macro-F1	Mono-lingual	Cross-lingual	Input tokens	Output tokens	Latency mean / p95
Advanced	0.940	0.951	0.934	5,576	58	2.1 s / 2.9 s
Beginner	0.975	0.984	0.971	2,523	55	1.1 s / 1.7 s

Evidence is given for every Entailment / Contradiction prediction; 78 % of the cited pages (advanced) and 100 % of the cited passages (beginner) lie inside the human-annotated reference section.

Advanced task by language pair (claim → booklet):

	→ de	→ fr	→ it
de	0.951	0.949	0.916
fr	0.825	0.956	1.000
it	0.926	0.952	0.934

5.2 Full document vs. selected context (advanced, same 150 test pairs)

Context	Prompt	Macro-F1	Input tokens	Output tokens	Latency p95
Full booklet	JSON	0.730	59,517	202	43.7 s*
Proposal filter, top 5 pages	JSON	0.795	3,408	157	36.2 s*
Proposal filter, top 10 pages	ids	0.880	6,502	54	4.1 s
Hybrid, top 10 pages	ids	0.926	6,023	55	3.8 s
Final (12 pages, clipping, speaker boost, decision rules)	ids	0.946	5,713	55	2.8 s

* measured while three runs shared the endpoint.

The full booklet is both the most expensive and the least accurate option: booklets reach 250k characters (~70k tokens), and the relevant passage gets lost in the middle. Selecting context is therefore not only an efficiency measure but also improves NLI quality.

5.3 Ablations on the dev split

Change	Effect	Decision
ids answer instead of JSON with reasoning + quotes	beginner 0.973 → 0.980, advanced 0.786 → 0.797, p95 5.5 s → 1.8 s	adopted
Hybrid retrieval instead of proposal filter	advanced 0.797 → 0.862	adopted
Viewpoint and recommendation rules in the prompt	advanced 0.862 → 0.870, beginner 0.980 → 0.993	adopted
Decision-Rule 0: keep the label when it disagrees with the confidences	44 fixed / 13 broken over all saved runs; test 0.920 → 0.928	adopted
Section labels from bare speaker lines and closing recommendation boxes	Federal Council pages labelled as committee 27 → 0; F1-neutral, evidence grounding 0.86 → 0.88	adopted
Clip pages > 3,000 characters to the best-matching window	test 0.925 → 0.928, input tokens 5,725 → 4,696 (p95 12.7k → 7.4k)	adopted
12 instead of 10 pages (after clipping)	all 1,495 pairs 0.912 → 0.931 (51 fixed / 24 broken); +~890 input tokens	adopted
Speaker boost (2.2)	all pairs 0.931 → 0.940 (22 fixed / 8 broken), committee-claim errors 36 → 27, no extra tokens	adopted
Decision-Rule 0b (2.3)	all pairs 0.940 → 0.945 (8 fixed / 1 broken)	adopted
600-character passages instead of pages (n = 450, paired)	0.831 vs. 0.908; better evidence precision, fewer tokens	rejected
Thinking before answering (prefilled reasoning marker)	dev 450: 0.908 → 0.929 (20 fixed / 11 broken); ~1,300 output tokens and ~23 s per claim instead of 59 and ~2 s; a 300-token budget is worse	rejected (efficiency)
Translate cross-lingual claims into the booklet language first	dev 0.908 → 0.919, test 0.925 → 0.923 (14 fixed / 11 broken); +0.6 s	rejected (noise)
Other fixes for speaker attribution: second verification pass, hiding the opposing side, few-shot examples, speaker hint in the prompt, flipping committee contradictions	each broke at least as many answers as it fixed (verification pass: 0 fixed / 24 broken)	rejected
Other answer formats and filters: label-only answer with logprobs, one-sentence reasoning, confidence thresholds, numerical override, meta-classifier	no gain, or worse (label-only: beginner 0.724); the meta-classifier's only useful split became Rule 0b	rejected
Apertus 8B instead of 70B (450 dev pairs)	0.851 vs. 0.908 F1, p95 1.2 s vs. 4.5 s	70B kept (quality first)

Full tables: [docs/experiments.md](#). Runs on 150 pairs carry about ± 3 F1 points of sampling noise; close decisions were re-run on 450 pairs.

5.4 What we learned about Apertus

- **Selected context beats more context.** With the whole booklet (~60k tokens) Apertus 70B reaches 0.73 macro-F1, with 10 selected pages 0.93. We found no position effect ("lost in the middle"): the full-document runs fail even when the answer is in the first third of the booklet, so the degradation is general distraction.
- **Apertus is over-confident.** Its self-reported confidences are 0 or 1 in 97–99 % of cases, and the probability of the label token is ~1.0 for wrong answers as well as for correct ones (41 wrong vs. 40 correct dev answers). Confidence thresholds cannot filter its errors.
- **Speaker attribution was its main weakness — mostly a retrieval problem.** Claims attributed to the committee failed in 13.4 % of cases (2.2 % for Federal Council claims), and no prompt change helped. The speaker boost and Rule 0b brought them down to 7.4 %, the same rate as claims without a speaker.
- **Thinking helps, at a high price.** Letting the thinking model reason before its answer fixes half of the remaining dev errors (+0.021 F1 on dev 450), but needs ~22× the output tokens and ~11× the latency. A prompt instruction alone does not make it think first; the reasoning marker has to be prefilled. Selective thinking saves little (it is needed on ~3/4 of claims), and a short hard budget is worse.
- **Cross-lingual asymmetry.** Over all 1,495 pairs, French claims against German booklets are the hardest pair (0.880); French claims against Italian booklets reach 0.986. Small samples mislead here: on the 43 test pairs it→de scored 0.80, over all 148 pairs 0.925.

- **8B vs. 70B.** Apertus 8B is $\sim 3.5\times$ faster (p95 1.2 s) but 6 F1 points weaker, mostly on cross-lingual pairs and contradictions.

6. Limitations

- **French claims against German booklets** are the weakest pair (0.880 over all pairs); their errors spread over many voting dates without a pattern, so no language-specific rule was added.
- **Remaining errors** over all 1,495 pairs (81): Entailment $\rightarrow$ Contradiction 35, Contradiction $\rightarrow$ Neutral 22, Contradiction $\rightarrow$ Entailment 11, Entailment $\rightarrow$ Neutral 8; in nearly all of them the gold page is in the context. Thinking fixes about half of such errors, at $\sim 11\times$ the latency.
- **Evidence precision:** with whole pages as passages, $\sim 75\%$ of cited pages lie inside the gold reference section. Smaller passages raise this to $\sim 90\%$ but cost F1 (5.3).
- **Layout:** section headings are detected from text lines and recommendation boxes; committee pages headed only by a slogan stay unlabelled. Tables and charts are read as plain text. Layout-aware parsing (Docling) was not adopted: the remaining errors are reasoning errors with the gold page in context, and it would add several GB of models to the image.
- **Latency** was measured client-side on a shared endpoint; the organisers measure it themselves.
- **Not political advice:** outputs describe the relationship between a claim and the official booklet only.

7. Reproducibility

```
export LLM_API_KEY="your_api_key_here"
make run          # Docker build + benchmark on the 402-pair test split (~4 min)
make test         # unit tests, no API calls

cd track_2a
python -m src.hf_dataset          # dataset, booklets, dev/test split
python -m src.benchmark -d data/hf/test.jsonl -t advanced # writes results/<timestamp>_...json
python -m src.benchmark -d data/hf/test.jsonl -t beginner
python -m src.results_summary     # results/summary.json (used by the app)
python -m src.warm-cache          # pre-parse booklets (disk cache keyed by file hash)
```

make run from a fresh clone reproduces the advanced test result exactly (macro-F1 0.940, commit e9f6cd7; temperature 0, so repeated runs differ by at most a few pairs).

Configuration is read from environment variables: LLM_NAME, LLM_BASE_URL, LLM_API_KEY, NLI_STRATEGY (hybrid), NLI_TOP_K (12), PAGE_MAX_CHARS (3000), PROMPT_MODE (ids), PASSAGE_CHARS (0 = pages).

8. Next steps

1. Error analysis of the remaining Entailment $\rightarrow$ Contradiction cases (35 of 81 errors), where the gold page is in the context.
2. Return a precise sentence within each cited page as evidence, keeping whole pages as model context.
3. Reasoning at lower cost: full thinking gains +0.02 F1 at $\sim 11\times$ latency; fine-tuning a smaller Apertus on the reasoning traces could keep the gain without the latency.
4. Multilingual dense retrieval (e.g. BGE-M3) and layout-aware parsing (e.g. Docling) — low priority: the remaining errors are mostly reasoning errors with the gold page retrieved.

License

Code: Apache License 2.0 ([LICENSE](#)). Documentation, including this report: Creative Commons Attribution 4.0 (CC-BY-4.0), as required by the Hack Apertus terms (section 6).